DO NOW — RECALL

Lesson 115 · The Stages of Compilation

Name: _____ Class: _____ Date: _____

1. What tool converts assembly mnemonics into machine instructions?

2. Why must a binary search operate on ordered data?

3. An OR gate has inputs 0 and 1. What is the output?

4. What is the effect of enqueue(X), enqueue(Y), dequeue() on an initially empty FIFO queue?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

An assembler.

2. Model answer

It discards half the search space using an ordering comparison.

3. Model answer

1.

4. Model answer

X is removed; Y remains at the front.

By the end, you can…

• Characters become tokens such as identifiers, numbers and operators.

• A parser checks grammar and builds structural representation.

• Target code is produced; valid transformations improve performance while preserving meaning.

Vocabulary — know these terms

lexer — Recognises tokens from character stream.

parser — Checks tokens against grammar.

code generation — Production of target instructions.

optimisation — Meaning-preserving transformation to improve an objective.

NEW LEARNING

Read, annotate and explain

Lexical analysis

Characters become tokens such as identifiers, numbers and operators.

Syntax analysis

A parser checks grammar and builds structural representation.

Code generation and optimisation

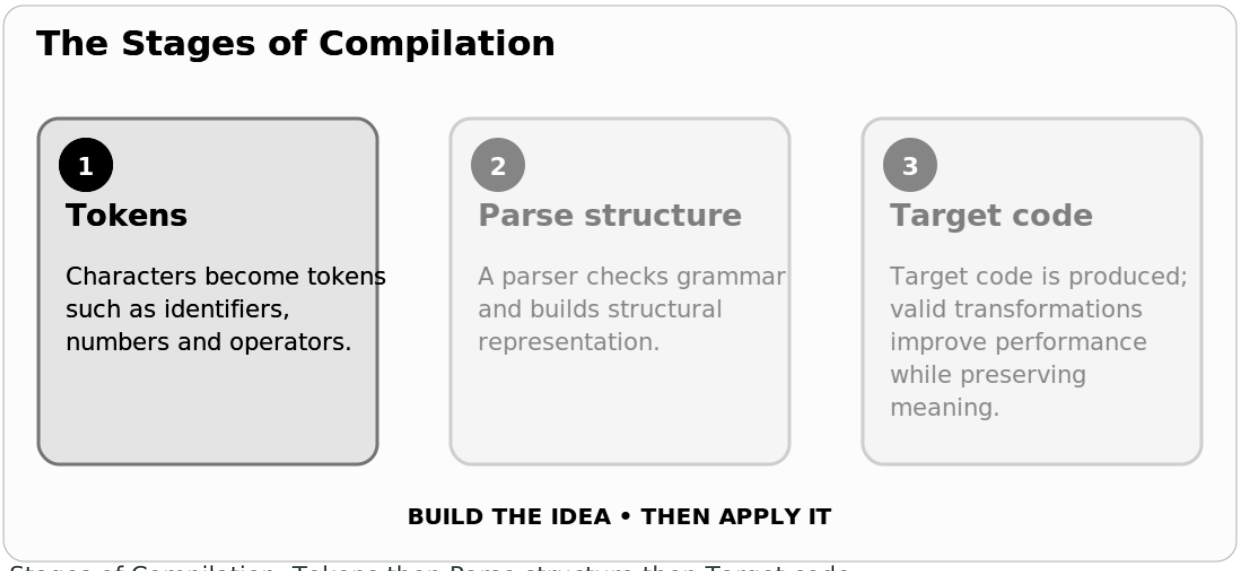
Target code is produced; valid transformations improve performance while preserving meaning.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

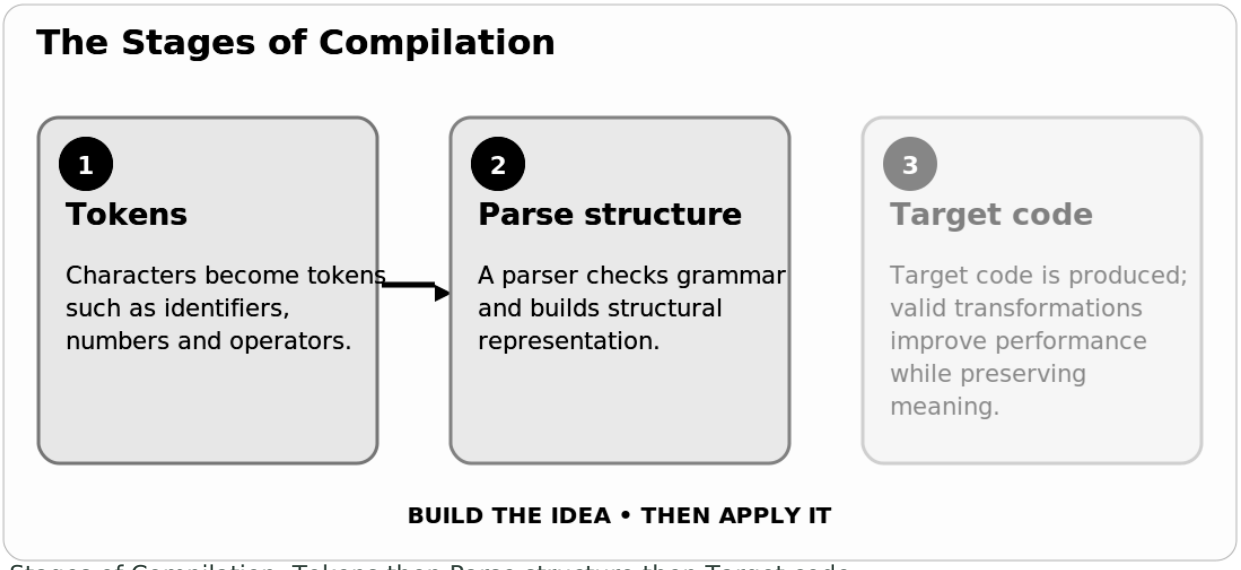
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



The Stages of Compilation: Tokens then Parse structure then Target code

New Learning

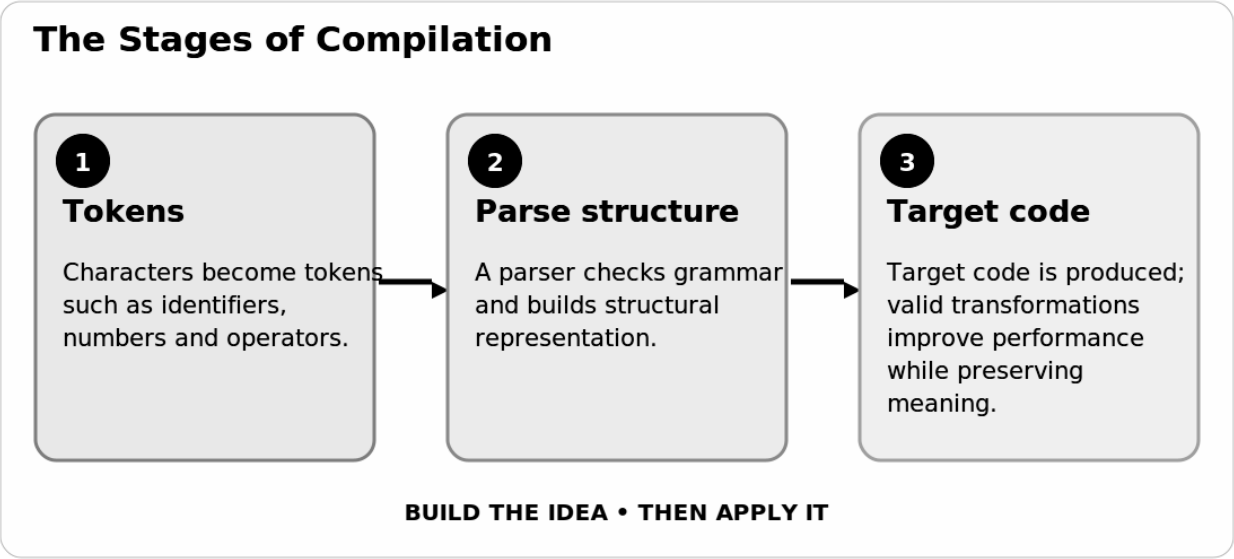


The Stages of Compilation: Tokens then Parse structure then Target code

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



The Stages of Compilation: Tokens then Parse structure then Target code

I DO — WORKED EXAMPLE

Follow the worked method

Explain the stages for the expression 'total = cost + 2' in a high-level source file.

Model answer

Lexer identifies identifier, assignment, identifier, plus, integer. Parser checks grammatical structure. The compiler creates suitable target instructions; optimisation might fold constants only where semantics permit.

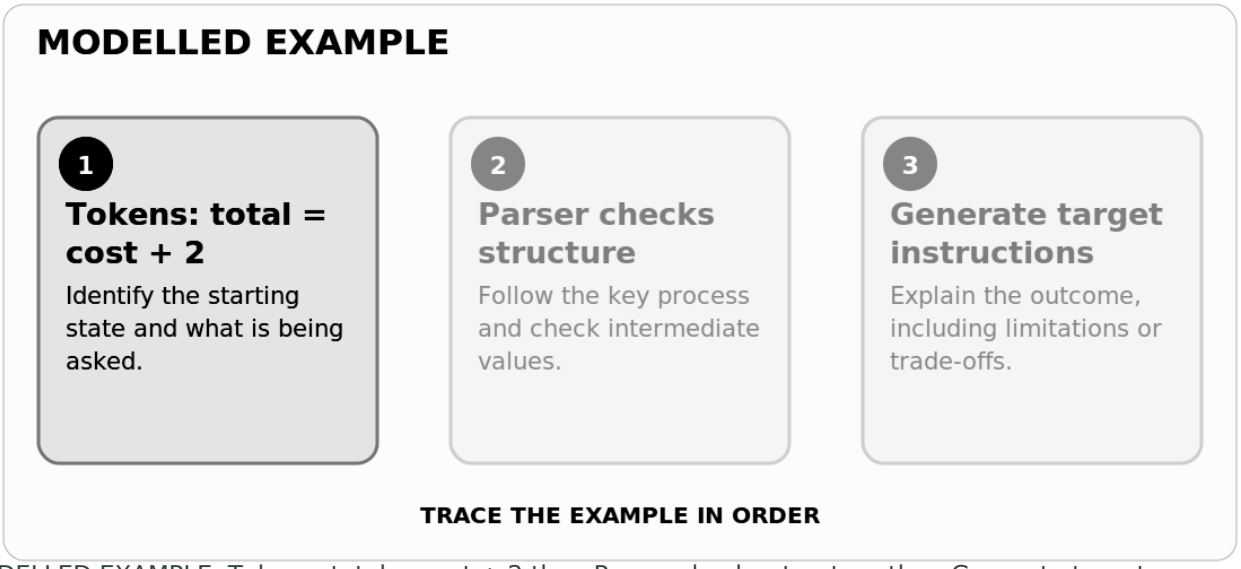
Check: link each claim to the scenario and use precise OCR terminology.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Explain the stages for the expression 'total = cost + 2' in a high-level source file.

Lexer identifies identifier, assignment, identifier, plus, integer. Parser checks grammatical structure. The compiler creates suitable target instructions; optimisation might fold constants only where



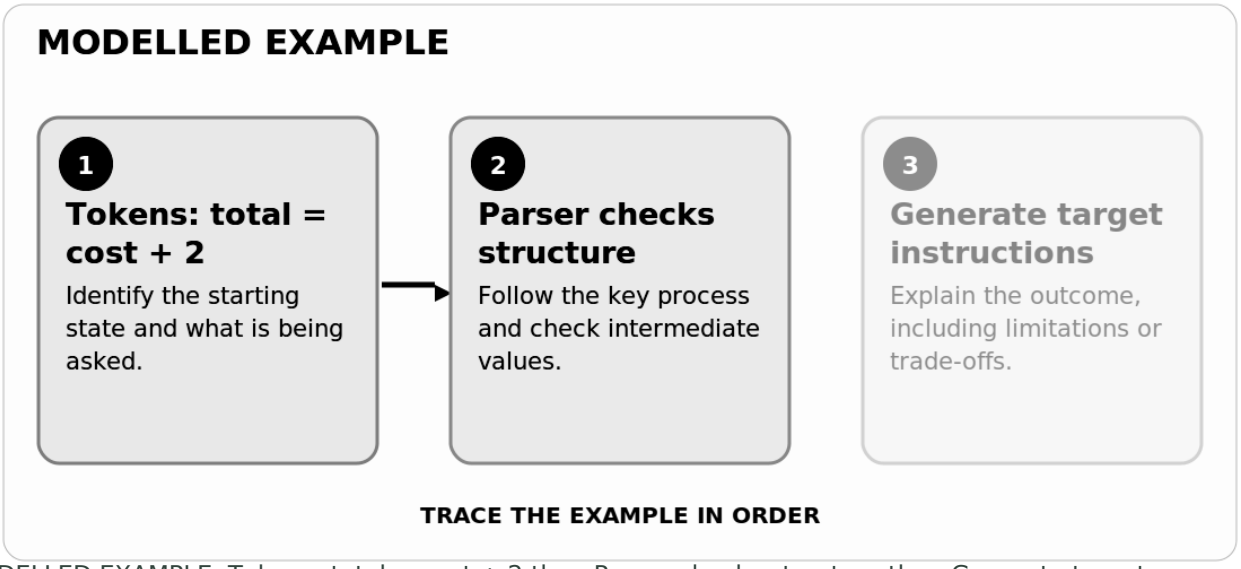
MODELLED EXAMPLE: Tokens: total = cost + 2 then Parser checks structure then Generate target instructions

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Explain the stages for the expression 'total = cost + 2' in a high-level source file.

Lexer identifies identifier, assignment, identifier, plus, integer. Parser checks grammatical structure. The compiler creates suitable target instructions; optimisation might fold constants only where



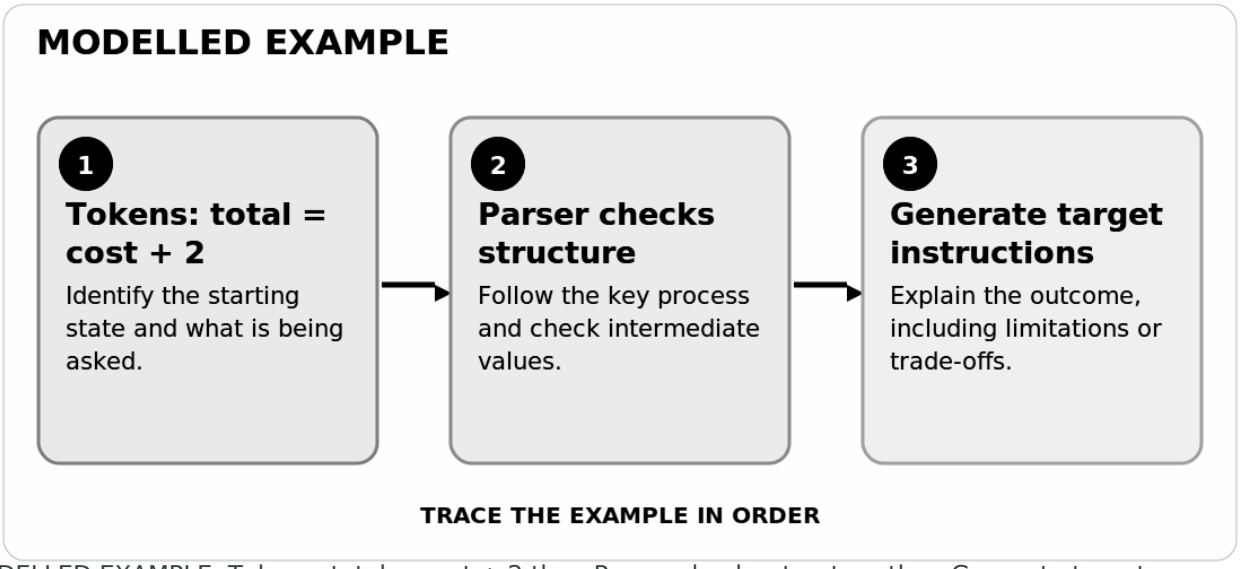
MODELLED EXAMPLE: Tokens: total = cost + 2 then Parser checks structure then Generate target instructions

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Explain the stages for the expression 'total = cost + 2' in a high-level source file.

Lexer identifies identifier, assignment, identifier, plus, integer. Parser checks grammatical structure. The compiler creates suitable target instructions; optimisation might fold constants only where



MODELLED EXAMPLE: Tokens: total = cost + 2 then Parser checks structure then Generate target instructions

WE DO — GUIDED PRACTICE

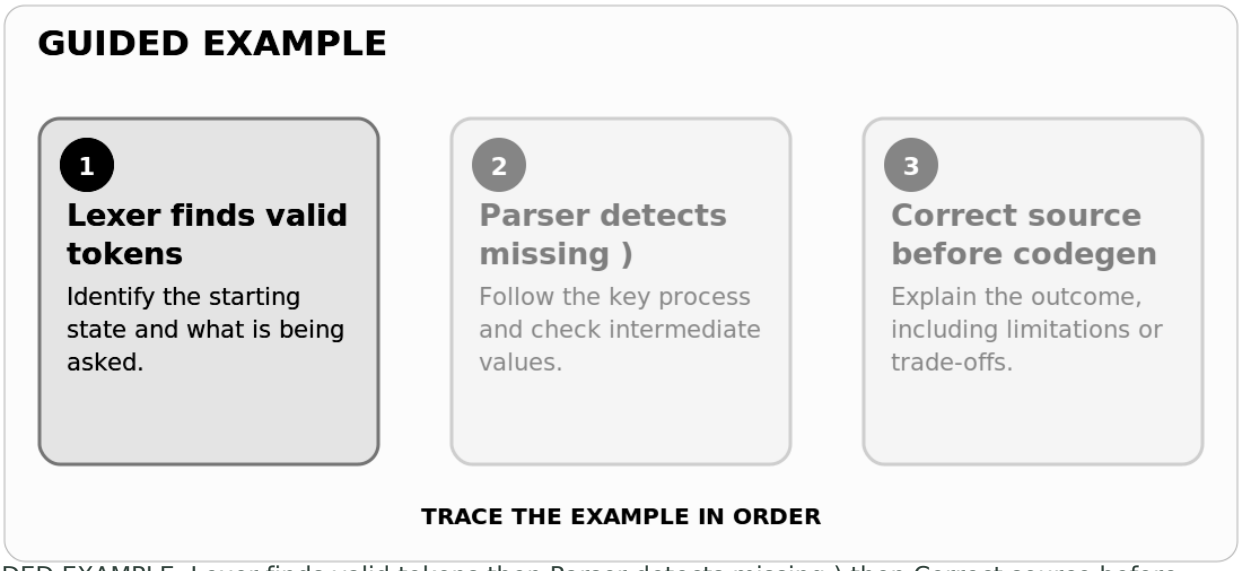
Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A programmer writes 'total = (cost + 2' without a closing bracket. Which stage finds the fault?

Identify the inputs, relevant facts and exact output or judgement required.



GUIDED EXAMPLE: Lexer finds valid tokens then Parser detects missing) then Correct source before codegen

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: What must we decide? Identify the inputs, relevant facts and exact output or judgement required.

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Describe lexical analysis, syntax analysis and code generation. [6]

Question 2

For 'score = score + 1', name five token categories or concrete tokens. [5]

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Evaluate whether an optimiser may replace 'x = a + 0' with 'x = a', considering language semantics. [8]

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

A syntax analyser is responsible primarily for reading characters into tokens.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

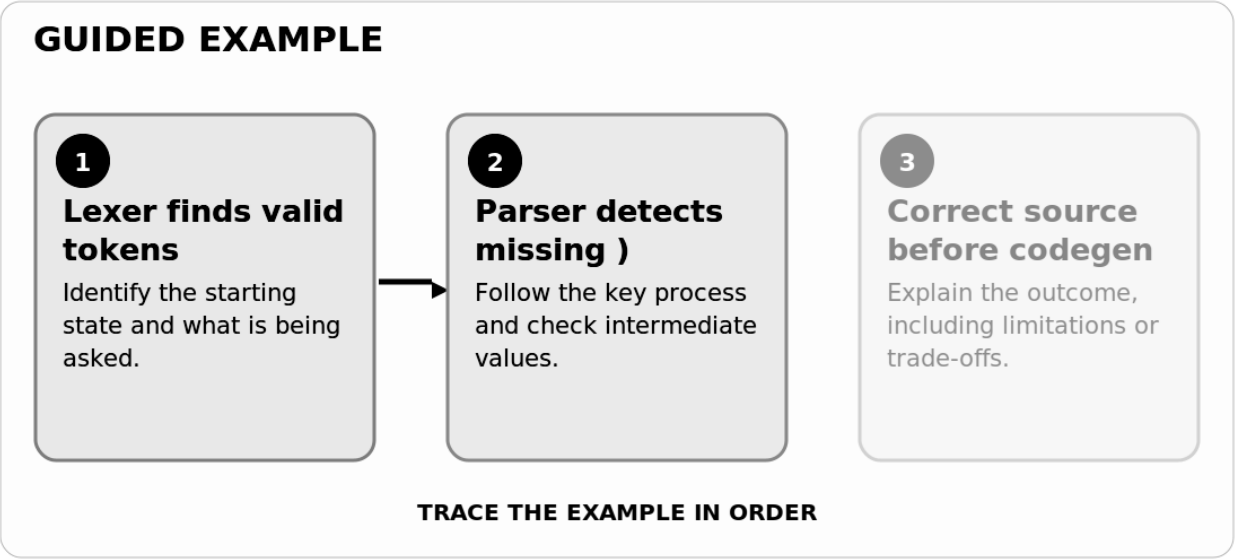
Next step: revisit the model or a relevant revision-guide section if needed.

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A programmer writes 'total = (cost + 2' without a closing bracket. Which stage finds the fault?

Identify the inputs, relevant facts and exact output or judgement required.



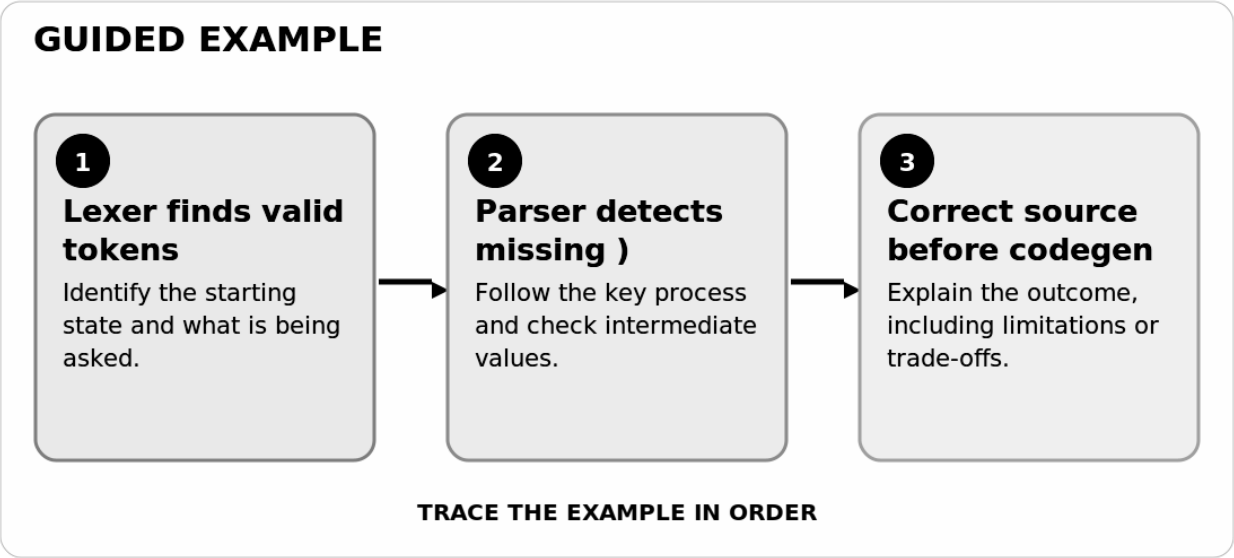
GUIDED EXAMPLE: Lexer finds valid tokens then Parser detects missing) then Correct source before codegen

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A programmer writes 'total = (cost + 2' without a closing bracket. Which stage finds the fault?

Identify the inputs, relevant facts and exact output or judgement required.



GUIDED EXAMPLE: Lexer finds valid tokens then Parser detects missing) then Correct source before codegen

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Work through the method

Use the middle stage of the visual to follow the process and state each intermediate result.

Shared worked answer

Lexical analysis can still recognise tokens. Syntax analysis detects a missing closing delimiter because token order violates grammar. No valid final executable follows until corrected.

IMPROVE: Explain one correction to your own reasoning.